

File-system Interface

Zhao Fang

Operating Systems
-for the collage students

Goal for Today

- File Concept
- Access Methods
- File Directories
- File-System Mounting(文件系统挂接)
- File Sharing
- Protection

Objective for this chapter

- To explain the function(功能) of file systems.
- To describe the interfaces(接口) to file systems.
- To discuss file-system design tradeoffs(设计的相关事情), including access methods, file sharing, file locking, and directory structures.
- To explore file-system protection.

Design Goals of File Systems

Physical reality	File system abstraction
Block-oriented(面向块的)	Byte-oriented(面向字节流)
Physical sectors(物理扇区)	Named files
No protection	Users protected from one another
Data might be corrupted if machine crashes(如果机器崩溃, 数据也会损坏)	Robust to machine failures(具有机器灾害环境下的鲁棒性)

Files

- Files are the central element to most applications(对大多数应用来说，文件是核心因素)
- The File System is one of the most important part of the OS to a user
- Desirable properties of files:
 - Long-term existence(长期存在)
 - Sharable between processes
 - Structure

File Concept

- Contiguous logical address space(连续逻辑地址空间)
- Types:
 - Data
 - numeric
 - character
 - binary
 - Program
- Contents defined by file's creator(内容由文件创建者定义)
 - Many types
 - Consider **text file, source file, executable file**

AI-Specific File Formats-for TensorFlow

TFRecord

Description

A binary file format natively optimized for TensorFlow, storing sequences of binary records to streamline I/O operations during model training and inference.

Key Advantages

Provides high throughput for large datasets, offers efficient storage compression, and seamlessly integrates with TensorFlow's data pipeline for fast ingestion.

AI Application

Standard packaging for training/validation datasets in TensorFlow, critical for distributed deep learning workloads.

AI-Specific File Formats-for HDF5

HDF5 (.h5)

Description

A hierarchical data format designed to store, organize, and share large and complex numerical datasets in a single, portable file.

Key Advantages

Native support for multi-dimensional arrays and rich metadata; lossless compression; widely compatible with scientific computing tools.

AI Application

Storing preprocessed sensor data, trained model weights, and high-dimensional feature embeddings for research reproducibility.

AI-Specific File Formats-for ONNX

ONNX (.onnx)

Description

An open standard format for representing deep learning models, enabling framework-agnostic model definition and cross-platform deployment.

Key Advantages

Eliminates vendor lock-in; supports hardware acceleration (GPUs/TPUs); reduces engineering effort for model conversion.

AI Application

Converting models between PyTorch/TensorFlow for production inference on edge devices, cloud services, or embedded systems.

File Management Systems

- Provides services to users and applications in the use of files
 - The way a user or application accesses files
- Programmer does not need to develop file management software

Objectives for a File Management System

- Meet the data management(数据管理) needs of the user
- Guarantee that the data in the file are valid(确保文件内的数据可用)
- Optimize performance(性能优化)
- Provide I/O support for a variety(各类) of storage device types
- Minimize lost or destroyed data(最小化数据的丢失和损坏)
- Provide a standardized set of I/O interface routines to user processes(向用户进程提供一套标准化的I/O接口例程)
- Provide I/O support for multiple users (if needed)

Requirements for a general purpose system

1. Each user should be able to create, delete, read, write and modify files
2. Each user may have controlled access to other users' files(每个用户可对其他用户的文件拥有受控制的访问权限)
3. Each user may control what type of accesses are allowed to the users' files(每个用户可控制对其文件允许的访问类型)
4. Each user should be able to restructure the user's files in a form appropriate to the problem(每个用户能针对问题以适合形式重新编排自己的文件)

Requirements cont.

5. Each user should be able to move data between files
6. Each user should be able to back up and recover the user's files in case of damage(每个用户应能备份和恢复自己的文件, 以防文件受损)
7. Each user should be able to access the user's files by using symbolic names(每个用户都能通过使用符号名称访问自己的文件)

File Operations in AI Workflows

Writing Files: Persisting AI Workflow Results

Critical for saving model checkpoints (periodic state snapshots for recovery), logging training metrics (loss, accuracy), and storing augmented synthetic data. This ensures that the expensive computational results of AI training are not lost and can be reused.

Reading Files: Ingesting Inputs and Models

Fundamental for loading raw datasets (training/validation splits) and pre-trained model weights. Efficient reading pipelines are essential to feed data into neural networks and initialize transfer learning models quickly.

Seeking: Efficient Big Data Navigation

Enables direct access to specific chunks of large binary files (e.g., TFRecords), avoiding loading entire datasets into memory and thus optimizing resource usage in large-scale AI training.

File Attributes: Basic Information

■ File Name

- Only information kept in human-readable form
- Name as chosen by creator (user or program).
- Must be unique within a specific directory(在特定目录内必须唯一).

■ Identifier(标识)

- Unique tag (number) identifies file within file system

■ File type

- needed for systems that support different types

File Attributes: Basic Information

- Location
 - Pointer to file location on device
- Size
 - Current file size
- File Organisation
 - For systems that support different organizations

File Attributes -continue

- Protection
 - Controls who can do reading, writing, executing
- Time, date, and user identification
 - Data for protection, security(安全), and usage monitoring
- Information about files are kept in the directory structure, which is maintained on the disk

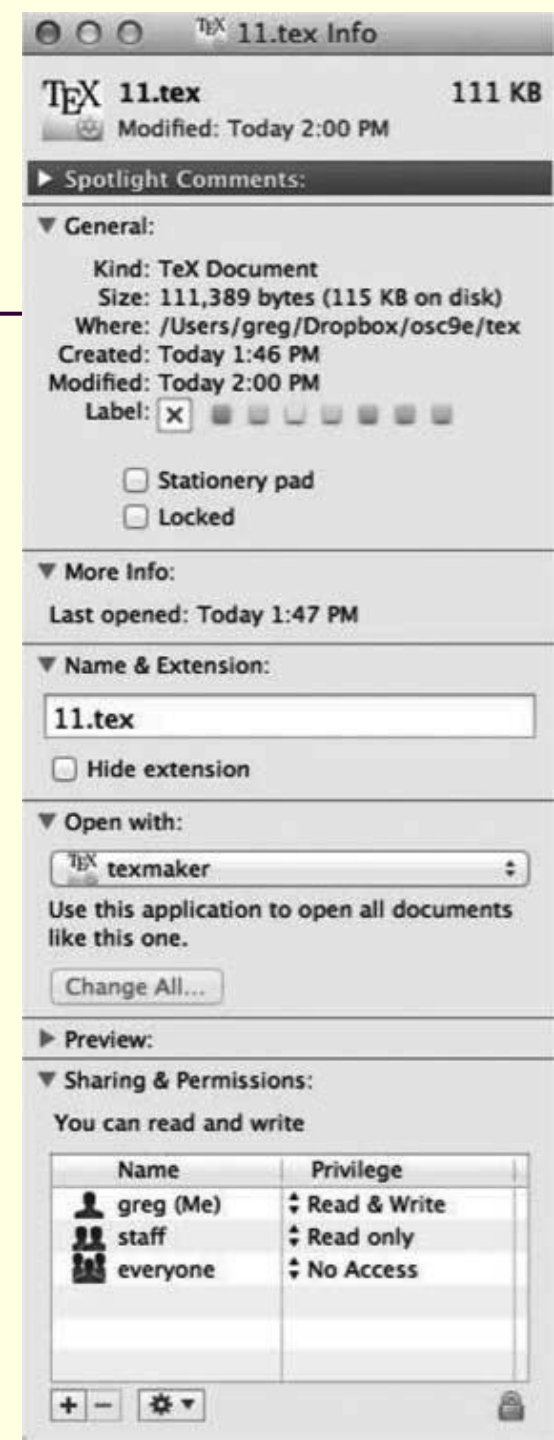
File Attributes - Address Information

- Volume(卷组)
 - Indicates device on which file is stored
- Starting Address
- Size Used
 - Current size of the file in bytes, words, or blocks
- Size Allocated
 - The maximum size of the file

File Attributes - Usage Information

- Date Created(创建时间)
- Identity of Creator(创建者标识)
- Date Last Read Access(最后读访问时间)
- Identity of Last Reader(最后读者标识)
- Date Last Modified(最后修改时间)
- Identity of Last Modifier(最后修改者标识)
- Date of Last Backup(最后备份时间)
- Current Usage
 - Current activity, locks, etc

File info Window on Mac OS X



Directory Elements: Access Control Information

- Owner(属主)
 - The owner may be able to grant/deny access to other users and to change these privileges(属主能授予 / 拒绝其他用户的访问权限).
- Access Information(访问信息)
 - May include the user's name and password for each authorized user(可能包含每个鉴权用户的用户名和密码).
- Permitted Actions
 - Controls reading, writing, executing, transmitting over a network

File Operations

- A file is an abstract data type.
- Create a file
 - Space in the file system must be found for the file(分配存储空间：需在文件系统中为该文件找到可用空间)
 - An entry for the new file must be made in the directory(创建目录：必须在目录中为新文件创建一个条目)
- Writing a file
 - Make a system call to specify the file name of the information that to be written(发起系统调用：指定要写入信息的文件名)
 - Search the directory to find the file 's location(搜索目录以找到该文件的存储位置)

File Operations

- **Read** – at **read pointer** location
- **Reposition within file**(文件内的定位) - **seek**
- **Delete**
- **Truncate**(截断)
- ***Open(F_i)*** – search the directory structure on disk for entry F_i , and move the content of entry to memory
- ***Close (F_i)*** – move the content of entry F_i in memory to directory structure on disk

Open Files

- Several pieces of data are needed to manage open files:
 - **Open-file table**: tracks open files(跟踪打开文件)
 - File pointer(文件指针): pointer to last read/write location, per process that has the file open(每个打开文件的进程所对应的上次读 / 写位置指针)
 - **File-open count**: counter of number of times a file is open – to allow removal of data from open-file table when last processes closes it(文件打开次数计数器 —— 用于当最后一个进程关闭文件时，允许从打开文件表中移除数据)

Open Files

- **Disk location of the file.** The information needed to locate the file on disk is kept in memory so that the system does not have to read it from disk for each operation(会保存文件在磁盘上的定位信息，这样就不必每次操作都从磁盘读取该信息).
- **Access rights.** Each process opens a file in an access mode. This information is stored on the per-process table so the operating system can allow or deny subsequent I/O requests(每个进程以某种访问模式打开文件。该信息存储在进程相关的表中，以便操作系统能够允许或拒绝后续输入 / 输出请求).

Open File Locking

- file systems
 - Similar to reader-writer locks
 - **Shared lock** similar to reader lock – several processes can acquire concurrently
 - **Exclusive lock(互斥锁)** similar to writer lock
- Provided by some operating systems and Mediates(协调) access to a file

Open File Locking

- Mandatory or advisory:
 - **Mandatory(强制性)** – access is denied depending on locks held and requested (访问是否被拒绝取决于已持有的锁和请求的锁)
 - **Advisory(劝告式)** – processes can find status of locks(锁的状态) and decide what to do

File Types – Name, Extension

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

File Structure

- None - sequence of words, bytes
- Simple record structure
 - Lines
 - Fixed length
 - Variable length
- Complex Structures
 - Formatted document(格式化文档是指经过排版和格式设置的文件，包含文本、图像、表格、样式、布局等结构化信息，其格式通常由特定的软件或规范定义，以确保内容在显示或打印时具有统一的外观和可读性)
 - Relocatable load file(可重定位加载文件是一种 目标代码文件，包含编译后的程序指令和数据)

File Structure

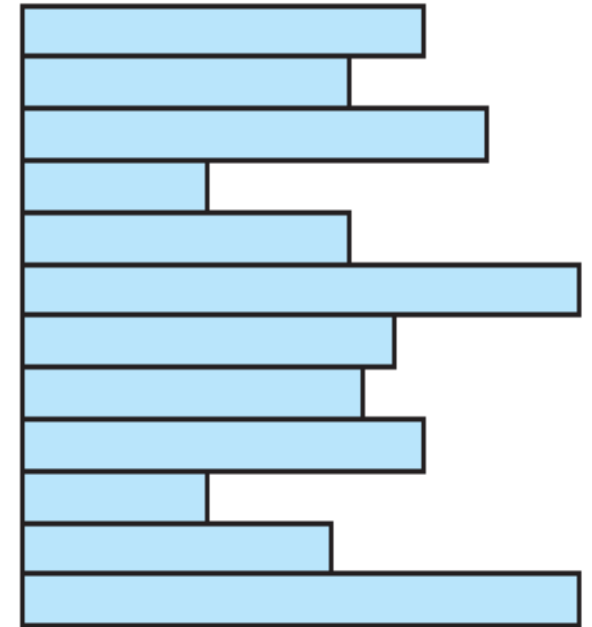
- Can simulate last two with first method by inserting appropriate control characters(通过在基础文件中插入具有特定含义的控制字符，可以模拟更复杂的文件结构（如格式化文档或可重定位文件))
- Who decides:
 - Operating system
 - Program

File Organisation Types

- Many exist, but usually variations of:
 - Pile(堆文件)
 - Sequential file(顺序文件)
 - Indexed sequential file(索引顺序文件)
 - Indexed file(索引文件)
 - Direct, or hashed, file(直接（哈希）文件)

The Pile

- Data are collected in the order they arrive
 - No structure
- Purpose is to accumulate a mass of data and save it(其目的是积累大量数据并将其保存下来)
- Records may have different fields
- Record access is by exhaustive search(记录访问通过穷举搜索实现)

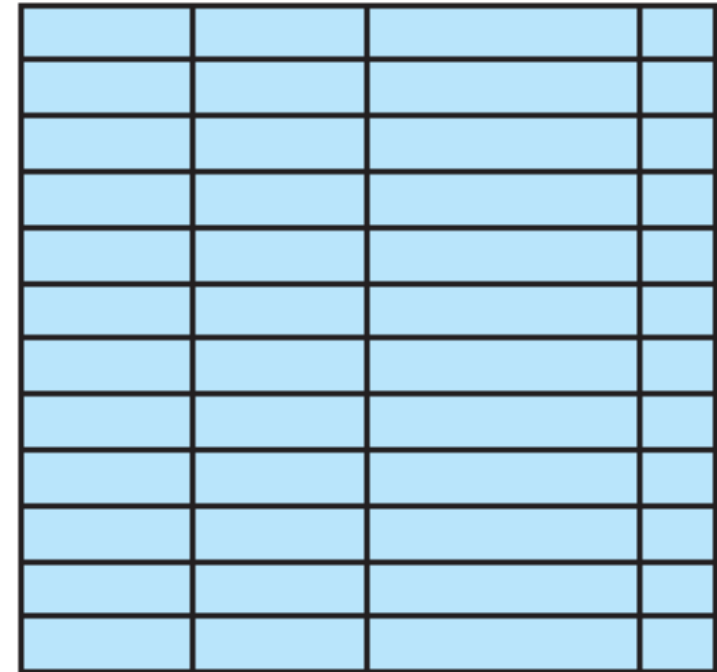


Variable-length records
Variable set of fields
Chronological order

(a) Pile File

The Sequential File

- Fixed format used for records
- Records are the same length
- All fields the same (order and length)
- Field names and lengths are attributes of the file
- Key field
 - Uniquely identifies the record
 - Records are stored in key sequence



Fixed-length records
Fixed set of fields in fixed order
Sequential order based on key field

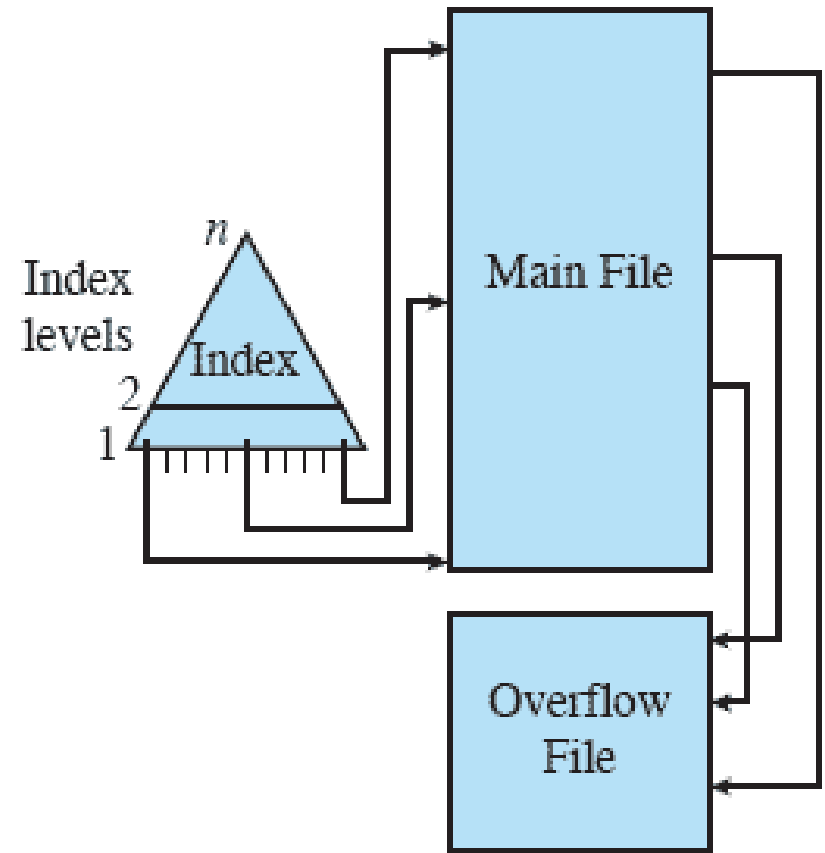
(b) Sequential File

Indexed Sequential File

- Maintains the key characteristic of the sequential file(保留了顺序文件的关键特征):
 - records are organized in sequence based on a key field.

Two features are added:

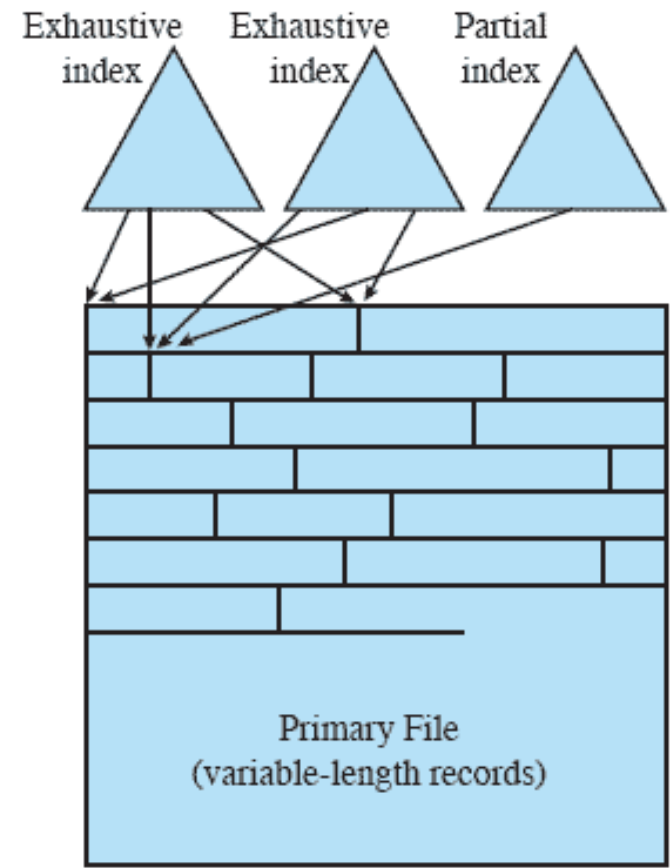
- an index to the file to support random access, and an overflow file.



(c) Indexed Sequential File

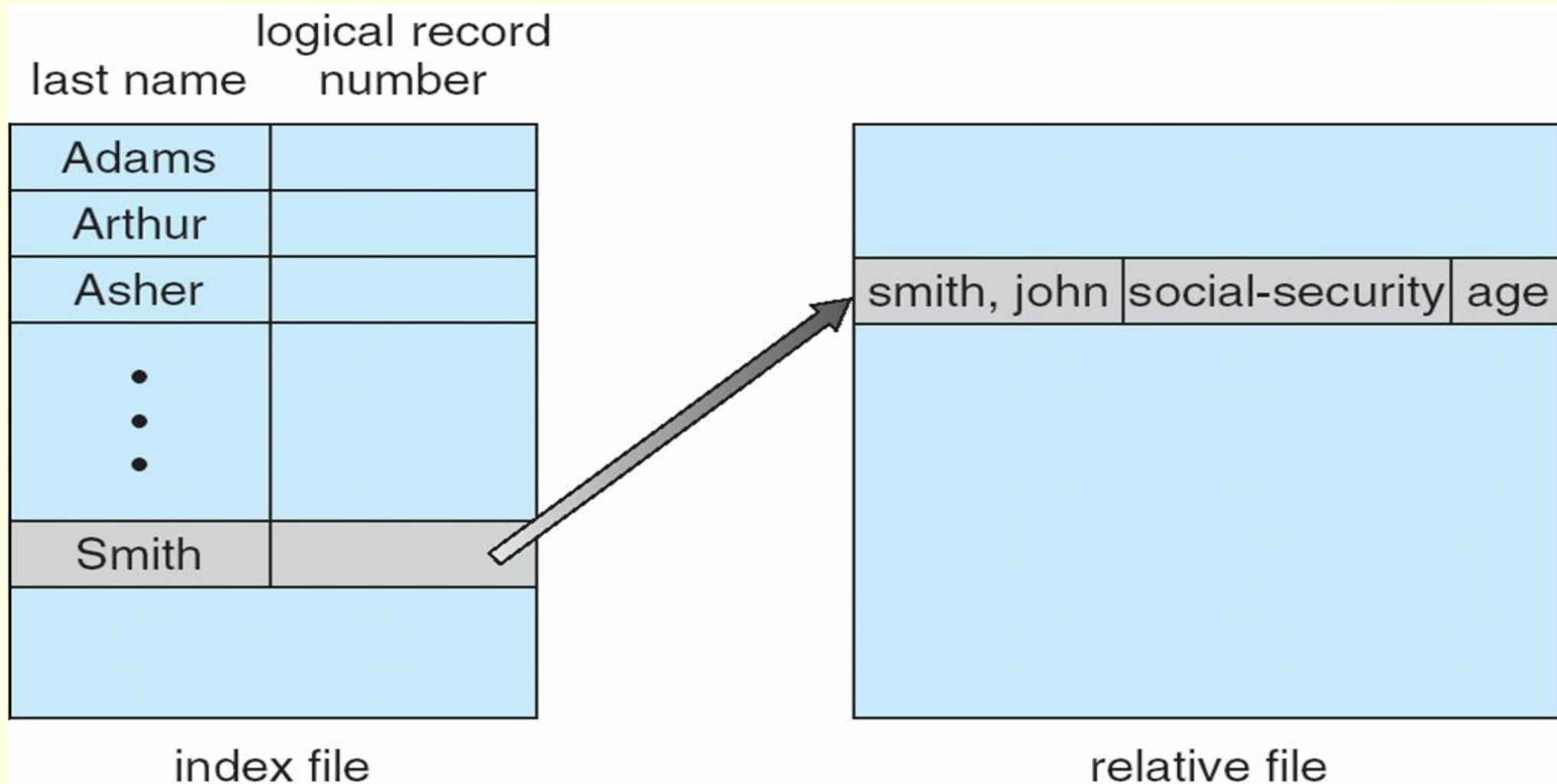
Indexed File

- Uses multiple indexes for different key fields
 - May contain an exhaustive index that contains one entry for every record in the main file(可包含一个详尽索引(其中包含主文件中每条记录的一个条目))
 - May contain a partial index
- When a new record is added to the main file, all of the index files must be updated.



(d) Indexed File

Example of Index and Relative Files



File Organization

- Access directly any block of a known address(可直接访问已知地址的任意数据块).
- The Direct or Hashed File(直接文件或哈希文件)
 - Directly access a block at a known address
 - Key field required for each record

Criteria for File Organization

- Important criteria include:
 - Short access time(访问时间短)
 - Ease of update(容易更新)
 - Economy of storage(存储经济)
 - Simple maintenance(维护简单)
 - Reliability(可靠)
- Priority will differ depending on the use (e.g. read-only CD vs Hard Drive)
 - Some may even conflict

Ways to Access a File

- People use file systems
- Design of file systems involves understanding how people use file systems
 - **Sequential access(顺序访问)**—bytes are accessed in order
 - **Random access (direct access)**—bytes are accessed in any order
 - **Content-based access(按内容访问)**—bytes are accessed according to constraints on byte contents
 - e.g., return 100 bytes starting with “aye carumba”

File Usage Patterns

- Most files are small, and most references are to small files
 - e.g., .login and .c files
- Large files use up most of the disk space
 - e.g., mp3 files
- Large files account for most of the bytes transferred between memory and disk(大文件占内存与磁盘之间传输字节的大部分)
- Bad news for file system designers

File System Design Constraints

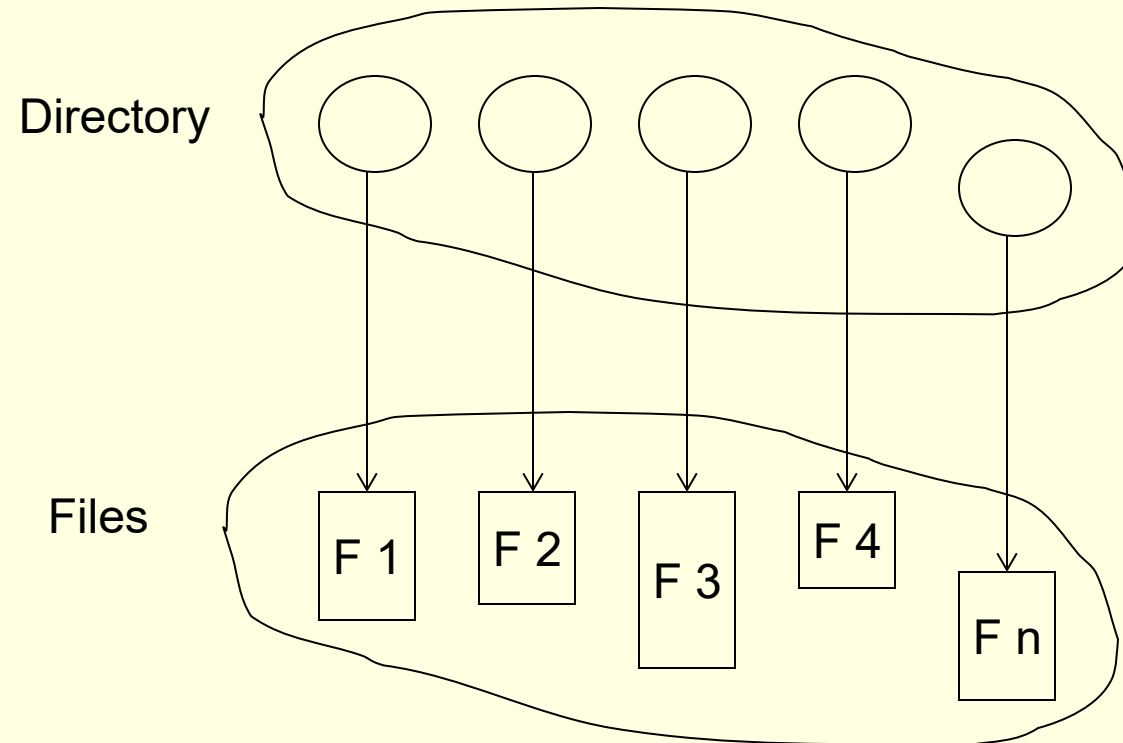
- High performance(高性能)
 - Efficient access of small files
 - Many small files
 - Used frequently
 - Efficient access of large files
 - Consume most disk space
 - Account for most of the data movement

Contents

- Contains information about files
 - Attributes
 - Location
 - Ownership
- Directory itself is a file owned by the operating system
- Provides mapping between file names and the files themselves(目录提供了文件名与文件本身之间的映射关系)

Directory Structure

- A collection of nodes containing information about all files



Both the directory structure and the files reside on disk

Operations Performed on a Directory

- A directory system should support a number of operations including:
 - Search(搜索)
 - Create files
 - Deleting files
 - Listing directory
 - Updating directory

Simple Structure for a Directory

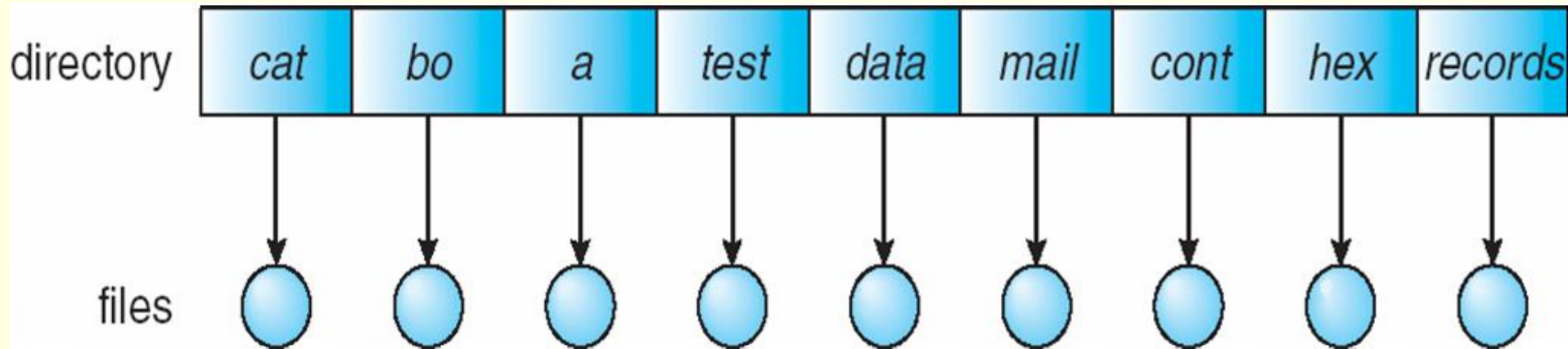
- The method for storing the previous information varies widely between systems
- Simplest is a list of entries, one for each file
 - Sequential file with the name of the file serving as the key
 - Provides no help in organizing the files
 - Forces user to be careful not to use the same name for two different files

Flat Name Space

- All files are stored in a single directory
- + Easy to implement
- Not scalable for large directories
 - **Name collisions(命名冲突)**: multiple files with the same names

Single-Level Directory

- A single directory for all users

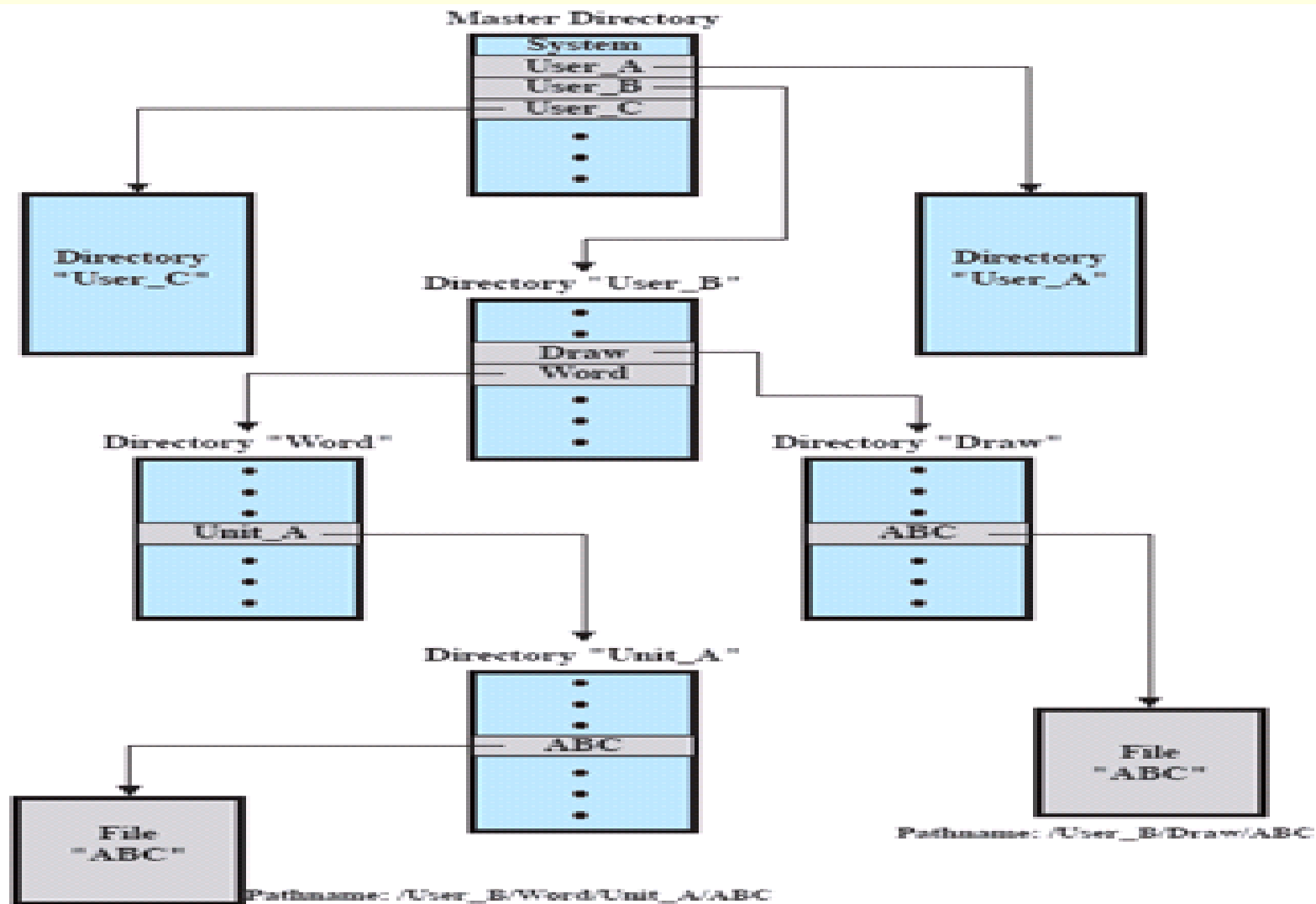


- Naming problem
- Grouping problem

Hierarchical Naming

- Uses multiple levels of directories
- Most popular name space organization
- + Conceptual model(概念模型) maps well into the human model of organizing things
 - A file cabinet(柜子) contains many files
- + Scalable(可扩展性)
 - The probability of name collisions decreases
- + Spatial locality(空间局部性)
 - Store all files under a directory within a cylinder to avoid disk seeks

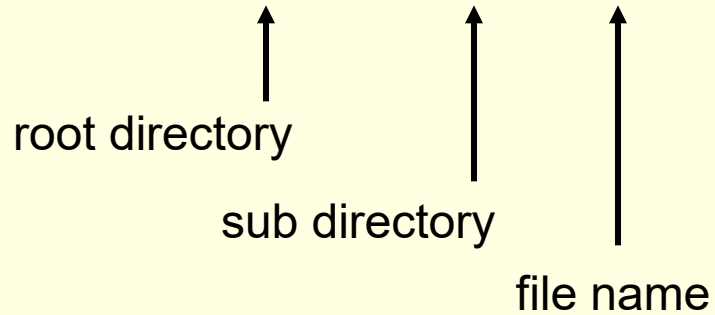
Directory Structure



More on Hierarchical Naming

- **Absolute path(绝对路径) name**: consisting the path from the **root directory** '/' to the file

- e.g., `/pets/cat.jpg`



An Example of Path Resolution

- To access the data content of `/pets/cat.jpg`
- The system needs to perform the following disk I/Os

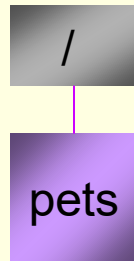
An Example of Path Resolution

- To access the data content of `/pets/cat.jpg`
- The system needs to perform the following disk I/Os
 1. Read in the file header for the root directory '/'
 - Stored at a fixed location on disk



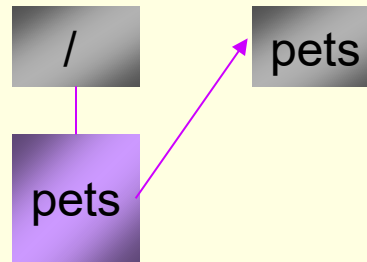
An Example of Path Resolution

- To access the data content of `/pets/cat.jpg`
- The system needs to perform the following disk I/Os
 2. Read the first data block for the root directory
 - Lookup the directory entry for `pets`



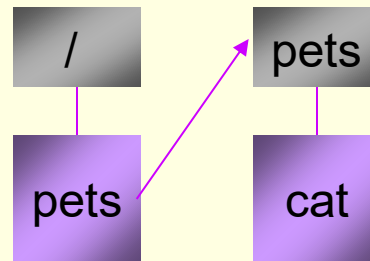
An Example of Path Resolution

- To access the data content of `/pets/cat.jpg`
- The system needs to perform the following disk I/Os
 3. Read the file header for `pets`



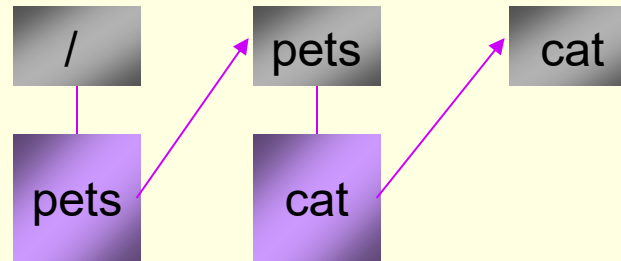
An Example of Path Resolution

- To access the data content of `/pets/cat.jpg`
- The system needs to perform the following disk I/Os
 4. Read the first data block for the `pet` directory
 - Lookup the directory entry for `cat.jpg`



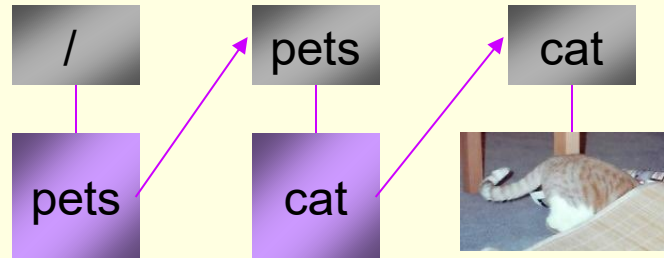
An Example of Path Resolution

- To access the data content of `/pets/cat.jpg`
- The system needs to perform the following disk I/Os
 5. Read the file header for `cat.jpg`



An Example of Path Resolution

- To access the data content of `/pets/cat.jpg`
- The system needs to perform the following disk I/Os
 6. Read the data block for `cat.jpg`

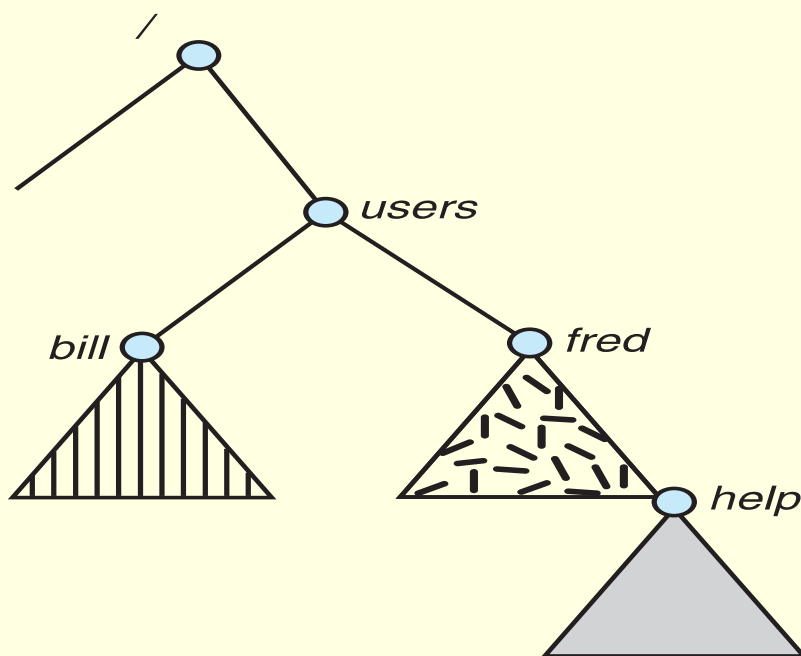


Working Directory

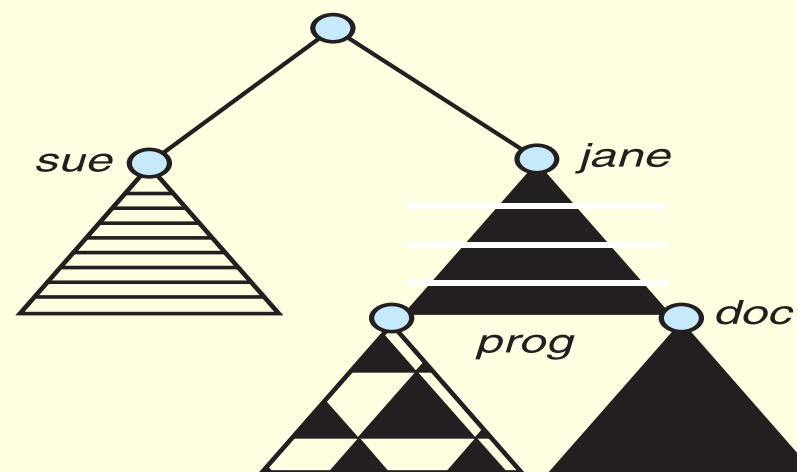
- Stating the full pathname and filename is awkward and tedious(陈述完整的路径名和文件名既麻烦又繁琐)
- Usually an interactive user or process is associated with a **current** or **working directory**
 - All file names are referenced as being relative to the working directory unless an explicit full pathname is used(除非使用显式的完整路径名，否则所有文件名均相对于工作目录引用)

File System Mounting

- A file system must be **mounted** before it can be accessed
- A unmounted file system is mounted at a **mount point**(挂载点)

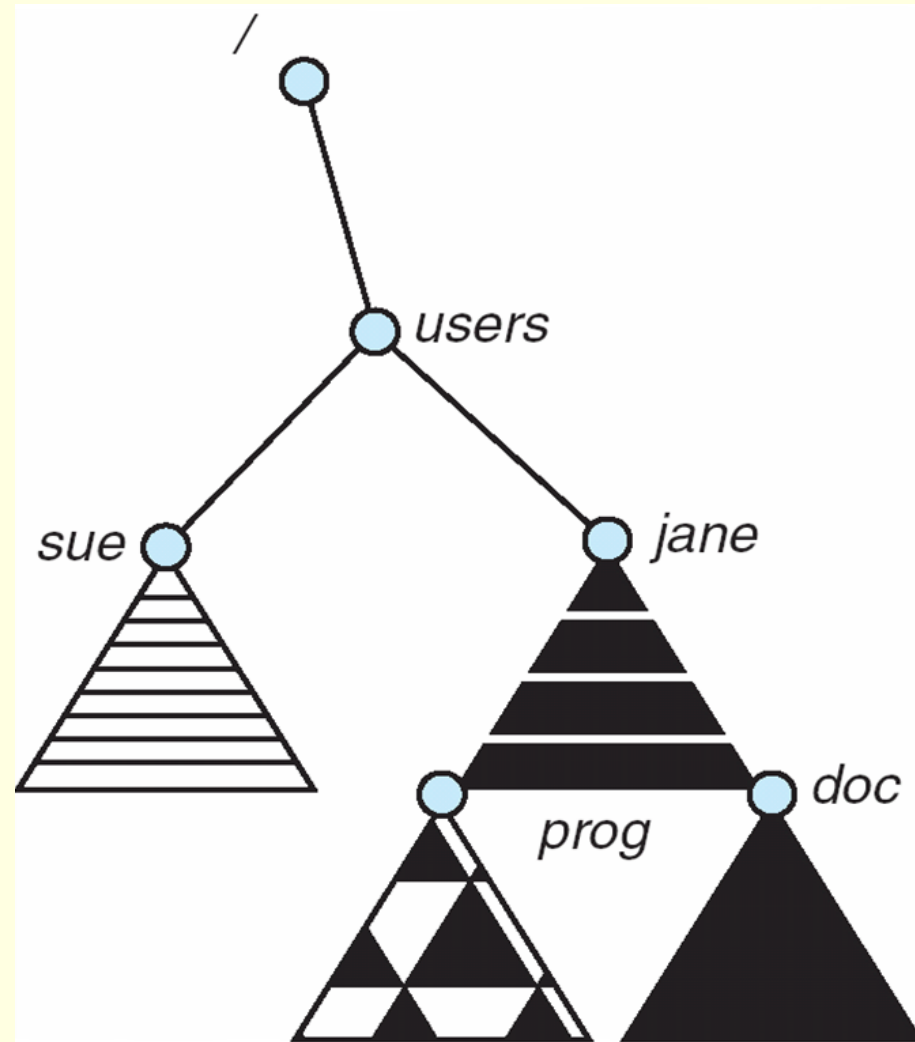


(a)



(b)

Mount Point



File Sharing

- Sharing may be done through a **protection** scheme
- On distributed systems, files may be shared across a network
- Network File System (NFS) is a common distributed file-sharing method
- In multiuser system, allow files to be shared among users
- Two issues
 - Access rights(访问权限)
 - Management of simultaneous access(并发访问管理)

File Sharing

- If multi-user system
 - **User IDs** identify users, allowing permissions and protections to be per-user
 - **Group IDs** allow users to be in groups, permitting group access rights
 - Owner of a file / directory
 - Group of a file / directory

Access Rights

- A wide variety of access rights have been used by various systems
 - often as a hierarchy where one right implies previous
- None
 - User may not even know of the files existence(用户甚至可能不知道这些文件的存在)
- Knowledge
 - User can only determine that the file exists and who its owner is(用户只能确定文件是否存在以及其所有者是谁)

Access Rights cont...

- Execution

- The user can load and execute a program but cannot copy it

- Reading

- The user can read the file for any purpose, including copying and execution

- Appending

- The user can add data to the file but cannot modify or delete any of the file's contents(用户可以向文件末尾添加新数据（如日志文件追加记录）)

Access Rights cont...

- Updating
 - The user can modify, delete, and add to the file's data.
- Changing protection
 - User can change access rights granted to other users
- Deletion
 - User can delete the file

User Classes

- Owner
 - Usually the files creator, usually has full rights
- Specific Users
 - Rights may be explicitly granted to specific users
- User Groups
 - A set of users identified as a group
- All
 - everyone

File Sharing for Distributed AI

AI Application: Distributed Training Challenge

In multi-GPU/server clusters, all worker nodes require concurrent access to massive datasets and evolving model files. A single point of failure or slow I/O can cripple the entire training pipeline, making specialized distributed storage a necessity.

HDFS

Optimized for high-throughput batch processing. Provides fault tolerance through data replication, ideal for large-scale training data ingestion.

NFS (Network FS)

A ubiquitous standard for low-latency file access over LANs. Simple to deploy and integrate with Linux-based AI clusters for smaller workloads.

Object Storage(S3)

Cloud-native, RESTful storage for unstructured data like model checkpoints. Offers infinite scalability and durability for cloud AI workflows.

Distributed File System (DFS) Pool

Unified, High-Bandwidth Data Source for All Nodes

GPU Node A

Worker Process
Reads Training
Data
& Model
Weights

GPU Node B

Parallel
Compute
Synchronizes
Gradients
With Parameter
Server

GPU Node C

Validation
Worker
Accesses
Shared
Checkpoint
Files

System Dynamics

By centralizing storage in a distributed file system, compute nodes avoid costly data duplication. The DFS handles concurrent read/write operations and data consistency, enabling linear scaling of training throughput as more GPU workers are added to the cluster.

Conclusion: A robust distributed file system is the invisible backbone of efficient large-scale AI. It synchronizes distributed compute resources, breaks down data silos, and ensures that training iterations are both fast and consistent across the entire cluster.

Simultaneous Access

- User may lock entire file when it is to be updated(用户可能在更新文件时锁定整个文件)
- User may lock the individual records during the update(用户可能在更新过程中锁定单个记录)
- Mutual exclusion and deadlock are issues for shared access(互斥和死锁是共享访问时需要解决的问题)

File Protection for AI Security

Access Control

Use OS-level permissions (e.g., Unix chmod) and Access Control Lists (ACLs) to enforce strict read/write/execute rules. This limits exposure to only authenticated and authorized users or AI service processes.

Data at Rest Encryption

Encrypt stored datasets and model binaries using strong algorithms. This renders data unreadable if physical or cloud storage is breached, safeguarding against offline theft of AI intellectual property.

Integrity Verification

Leverage cryptographic hashing to generate unique fingerprints. Regularly verify these hashes to detect tampering, ensuring model integrity and trustworthiness during deployment.

Key Takeaway: The First Line of Defense

AI models and training data represent irreplaceable digital assets. File system security is not just a technicality but a critical component of AI governance. Implementing robust file-level controls directly mitigates existential risks like model theft, data leakage, and adversarial manipulation of core AI infrastructure.

AI Asset Protection Workflow



Define System Permissions

Establish baseline ownership and chmod flags to restrict raw access to sensitive AI files at the operating system level.



Enforce Granular ACLs

Extend beyond basic permissions to manage specific user and group privileges, allowing only authorized workflows to interact with critical data.



Validate & Monitor Integrity

Implement continuous hash verification to confirm assets are unaltered, closing the loop on trust for production AI deployments.

Assignments

- Practice Exercises

- 11.2 11.4 11.5 11.6

- Exercises

- 11.10 11.11 11.12 11.15 11.17